

??:

```
'''bash
python -m venv .venv
source .venv/bin/activate # Windows? .venv\Scripts\activate
pip install -r requirements.txt
cp config.example.yaml config.yaml
'''
```

3.2 Windows + ????(??/????, ??/??)

- ?? OpenAPI+? ????? Windows ??? ?????.
- HTS/???? ??, API ??, ?????, ??? ?? ??? ?????.
- ? ??? ??? **??? ?????**? ?????, ?? ?? ?? ??? 'pykiwoom' ????? ?? ????? ???.

3.3 Mac

- ??(???) ?? ?????/????/??? Mac??? ??? ??
- ?? OpenAPI ????? ?? Windows VM ?? ?? Windows ?? ??

4. ??? ??/??/???? ??

4.1 ??? ??

- ??: OHLCV (??/??/??/??/???)
- ??: ?????, ?????, ???(????), ?????, ?? ??

4.2 ??

- ??? ??(KST/UTC ?? ??)
- ??? ?? ?? ??(????/??)
- ??? ?? ??(???? ???)
- ?????/??/???? ??(??)

4.3 ?????? ??? ??? ??

- ???
- ?????
- ?????(????)
- ???(????)
- ??/????(????)

4.4 ?? ??? ???

1. **???? ?????**: ?? ??? ?? ??
2. **????? ?????**: ?? ?? ??
3. **????(????)**: ????? ??? + ??????
4. **??? ???**: ?? ?? ?? ??? ??

5. ?? ?? ?? (???? ?? ?? ??)

??? ??? ??? ?? ?????. ? ??? ??/??/???? ?? ??? ?????.

5.1 ???? (Trend Following)

- ???? : ??? ? ??, ??? ? ??
- ?? : ???? ??, ?? ??, ADX ??
- ??? : ??? ??(????)
- ???? : ??? ????? + ?? ??

5.2 ???? (Mean Reversion)

- ???? : ??? ??? ????? ??
- ?? : ?????, RSI ???, z-score
- ??? : ??? ????? ? ??
- ???? : ????? ?? ??

5.3 ?????? / ???

- ???? : ??? ?? ? ?? ??
- ?? : N? ?? ??, ??? ??
- ??? : ??? ??????
- ???? : ??? ?? ?? ??

5.4 ?????? ??? (??/????)

- ???? : ????? ?? ?? ??
- ?? : ??/?? ??? ?? + ?????
- ??? : ?? ??(?? ??)
- ???? : ????? ?? ??

5.5 ??/??? ?? (Stat-Arb)

- ???? : ??? ??? ????? ??
- ?? : pair selection + z-score spread
- ??? : ??? ?? ??
- ???? : ?????/??? ?? ??

5.6 ?????/??

- ???? : ?? ????? ??? ??
- ?? : ???? , ???? , ????
- ??? : ??????
- ???? : ????? ??

5.7 ??? ???

- ???? : ??, ??, ????? ? ??? ??
- ?? : ??? ??? + ?????
- ??? : ????? ??

- ????: ????? ????

5.8 ?? ?? ??

- ????: ?????(IV) vs ?????(RV) ??
- ??: ????, ???/?????, ?? ????
- ????: ? ???, ?? ??
- ????: ???(Delta/Gamma/Vega/Theta) ???

5.9 ?? ??/??-?? ????

- ????: ????(???/?????) ??
- ??: ??? ??, ??????
- ????: ???/??? ???
- ????: ?? ?? ??

5.10 ????? ??

- ??? ??, ??? ???? , ??? ???
- ????: ???/???/???/??
- ????: ??? ?? ??

5.11 ???/??????

- ?????, ????, ????
- ????: ??(latency), ???, ???
- ????: ????? ??? ?? ?? ??

5.12 ML/??? ??

- ??: ??? + ??? ?? ??
- ????: ??? ??, ????
- ????: ?? ??? ?? PnL/??? ??

6. ???/???/??? ??/??

6.1 ????

- ???/???/??? ?? ??
- ?? ?? ? ??? ?? ??
- ??? ???? ??

6.2 ????

- API ????
- ?? ??
- ??? ????
- ??? ??

6.3 ??? ??

- ???/??/??? ?? DB? ??
- ????? ??? ? DB ????? ??
- ?? ? ??? ?? ??? ??(reconcile)

6.4 ??/??

- ?? ??, ?? ??, ?? ?? ??
- ?????/?? ??
- ????? ? ?? ??

6.5 ?????

- ?? ??('KILL_SWITCH') ?? ? ????? ??
- ??? ?? ?? ? ?? ??
- ??? ?? ?? ?? ? ?????

7. ? ????? ?? ???

7.1 ?? ??

- 'trading_system.py': ??/??/????/???/?? ??
- 'config.example.yaml': ?? ???
- 'requirements.txt': ??? ??

7.2 ?? ??

```
'''bash
pip install -r requirements.txt
cp config.example.yaml config.yaml
python trading_system.py
'''
```

7.3 ????? ??

```
'''bash
BACKTEST_ONLY=1 python trading_system.py
'''
```

7.4 ??? ??

1. 'config.yaml'? 'runtime.mode: live'? ??
2. ??? ? ??
3. ?? ?? ?? ?? ? ???
4. ??/??? ??? ??

8. ?? ?? ??? (?? ? ??)

```
1??(1~2?): Python ?? + ?? ??/?? ??
2??(2~4?): ??? ???? ?? + ??? ????
3??(4~8?): ??? ????? + ?? ??? ?
4??(2~3?): ??? ???/??/?? ??
5??(?): ????, ???, ?? ?? ???
```

```
## 9. ??? ???? (?? ??)
```

```
### 9.1 ???/???/??/? (??)
```

```
- ?? ?? ??, ??/?? ?? ??, ??? ??? ?? ??
- ??/??? ??/???/??? ??
- API ?? ??/?? ?? ?? ??
```

```
### 9.2 ?? ??/? (??? ??)
```

```
- 24/7 ????? ?? ??/??? ?? ??? ??
- ??? ?? ???? ?? ??
- ??? ???(??/???/???) ??
```

```
## 10. ??? ?????
```

```
- [ ] ???? ?? ?? ??
- [ ] ???/???/?? ??
- [ ] ??? ??? ?? ??
- [ ] ???/??? ???? ??
- [ ] ??? ?? + ?? ?? ??
- [ ] ???/? ???? ??
- [ ] ?? ???? ???? ??
```

```
# ?? A. ?? ?? ??
```

```
## A-1) 'trading_system.py'
```

```
'''python
#!/usr/bin/env python3
'''
???? ???? ???? ??? (?? + ?? ??? ???)
- ?? ?? : PAPER(??? ???? )
- ?? ???? LIVE(???) ?? ??
- ?? ??? : ??/????, ??, ??, ?? ??/?(??? ??)

?:
- ? ??? '?? ?? ? ???'???.
- ??? ?? ??? ????/????/? ???? ?????.
'''
```

```

from __future__ import annotations

import dataclasses
import datetime as dt
import enum
import hashlib
import hmac
import json
import logging
import math
import os
import sqlite3
import sys
import threading
import time
import traceback
import uuid
from abc import ABC, abstractmethod
from collections import deque
from pathlib import Path
from typing import Any, Deque, Dict, Iterable, List, Optional, Tuple

import requests
import yaml

# -----
# 1) ?? ??? ??
# -----

class Mode(str, enum.Enum):
    PAPER = "paper"
    LIVE = "live"

class AssetClass(str, enum.Enum):
    KR_STOCK = "kr_stock"
    US_STOCK = "us_stock"
    FUTURES = "futures"
    OPTIONS = "options"
    CRYPTO_SPOT = "crypto_spot"
    CRYPTO_FUTURES = "crypto_futures"

class Side(str, enum.Enum):
    BUY = "buy"
    SELL = "sell"

@dataclasses.dataclass
class Candle:

```

```
ts: dt.datetime
open: float
high: float
low: float
close: float
volume: float
```

```
@dataclasses.dataclass
class Signal:
    symbol: str
    asset_class: AssetClass
    side: Side
    confidence: float
    reason: str
```

```
@dataclasses.dataclass
class Position:
    symbol: str
    asset_class: AssetClass
    qty: float
    avg_price: float
```

```
@dataclasses.dataclass
class OrderRequest:
    symbol: str
    asset_class: AssetClass
    side: Side
    qty: float
    order_type: str = "market"
    price: Optional[float] = None
```

```
@dataclasses.dataclass
class FillEvent:
    order_id: str
    symbol: str
    asset_class: AssetClass
    side: Side
    qty: float
    price: float
    fee: float
    ts: dt.datetime
```

```
# -----
# 2) ??/??/?? ??
# -----
```

```
@dataclasses.dataclass
```

```

class RuntimeConfig:
    mode: Mode
    base_ccy: str
    loop_interval_sec: int
    max_daily_loss_pct: float
    max_position_pct: float
    kill_switch_file: str
    telegram_token: str
    telegram_chat_id: str

@dataclasses.dataclass
class BrokerConfig:
    upbit_access_key: str
    upbit_secret_key: str
    kiwoom_enabled: bool

@dataclasses.dataclass
class UniverseItem:
    symbol: str
    asset_class: AssetClass
    strategy: str

@dataclasses.dataclass
class Config:
    runtime: RuntimeConfig
    broker: BrokerConfig
    universe: List[UniverseItem]

class ConfigLoader:
    @staticmethod
    def load(path: str) -> Config:
        with open(path, "r", encoding="utf-8") as f:
            raw = yaml.safe_load(f)

            runtime = RuntimeConfig(
                mode=Mode(raw["runtime"]["mode"]),
                base_ccy=raw["runtime"].get("base_ccy", "KRW"),
                loop_interval_sec=int(raw["runtime"].get("loop_interval_sec", 30)),
                max_daily_loss_pct=float(raw["runtime"].get("max_daily_loss_pct", 2.0)),
                max_position_pct=float(raw["runtime"].get("max_position_pct", 10.0)),
                kill_switch_file=raw["runtime"].get("kill_switch_file", "KILL_SWITCH"),
                telegram_token=raw["runtime"].get("telegram_token", ""),
                telegram_chat_id=raw["runtime"].get("telegram_chat_id", ""),
            )

            broker = BrokerConfig(
                upbit_access_key=raw["broker"].get("upbit_access_key", ""),
                upbit_secret_key=raw["broker"].get("upbit_secret_key", ""),

```

```

        kiwoom_enabled=bool(raw["broker"].get("kiwoom_enabled", False)),
    )

    universe = [
        UniverseItem(
            symbol=i["symbol"],
            asset_class=AssetClass(i["asset_class"]),
            strategy=i["strategy"],
        )
        for i in raw.get("universe", [])
    ]

    return Config(runtime=runtime, broker=broker, universe=universe)

def setup_logger(log_path: str = "trading.log") -> logging.Logger:
    logger = logging.getLogger("algo")
    logger.setLevel(logging.INFO)
    logger.handlers.clear()

    fmt = logging.Formatter("%(asctime)s | %(levelname)s | %(message)s")

    fh = logging.FileHandler(log_path, encoding="utf-8")
    fh.setFormatter(fmt)
    logger.addHandler(fh)

    sh = logging.StreamHandler(sys.stdout)
    sh.setFormatter(fmt)
    logger.addHandler(sh)

    return logger

class StateStore:
    def __init__(self, db_path: str = "state.db"):
        self.conn = sqlite3.connect(db_path, check_same_thread=False)
        self._init_tables()
        self.lock = threading.Lock()

    def _init_tables(self) -> None:
        c = self.conn.cursor()
        c.execute(
            """
            CREATE TABLE IF NOT EXISTS positions (
                symbol TEXT,
                asset_class TEXT,
                qty REAL,
                avg_price REAL,
                PRIMARY KEY(symbol, asset_class)
            )
            """
        )

```

```

c.execute(
    """
    CREATE TABLE IF NOT EXISTS fills (
        order_id TEXT,
        symbol TEXT,
        asset_class TEXT,
        side TEXT,
        qty REAL,
        price REAL,
        fee REAL,
        ts TEXT
    )
    """
)
c.execute(
    """
    CREATE TABLE IF NOT EXISTS kv (
        k TEXT PRIMARY KEY,
        v TEXT
    )
    """
)
self.conn.commit()

def upsert_position(self, p: Position) -> None:
    with self.lock:
        self.conn.execute(
            "INSERT OR REPLACE INTO positions(symbol, asset_class, qty, avg_price) VALUES(?,?,?,?)"
            (p.symbol, p.asset_class.value, p.qty, p.avg_price),
        )
        self.conn.commit()

def get_positions(self) -> Dict[Tuple[str, AssetClass], Position]:
    rows = self.conn.execute("SELECT symbol, asset_class, qty, avg_price FROM positions").fetchall()
    return {
        (r[0], AssetClass(r[1])): Position(symbol=r[0], asset_class=AssetClass(r[1]), qty=r[2], avg
        for r in rows
    }

def add_fill(self, fill: FillEvent) -> None:
    with self.lock:
        self.conn.execute(
            "INSERT INTO fills VALUES(?,?,?,?,?,?,?,?)",
            (
                fill.order_id,
                fill.symbol,
                fill.asset_class.value,
                fill.side.value,
                fill.qty,
                fill.price,
                fill.fee,
                fill.ts.isoformat(),
            )

```

```

        ),
    )
    self.conn.commit()

# -----
# 3) ???/??? ???
# -----

class MarketDataAdapter(ABC):
    @abstractmethod
    def get_recent_candles(self, symbol: str, asset_class: AssetClass, n: int = 200) -> List[Candle]:
        ...

class BrokerAdapter(ABC):
    @abstractmethod
    def submit_order(self, req: OrderRequest) -> FillEvent:
        ...

class UpbitAdapter(MarketDataAdapter, BrokerAdapter):
    BASE_URL = "https://api.upbit.com"

    def __init__(self, access_key: str, secret_key: str):
        self.access_key = access_key
        self.secret_key = secret_key

    def get_recent_candles(self, symbol: str, asset_class: AssetClass, n: int = 200) -> List[Candle]:
        # symbol ??: KRW-BTC
        if asset_class not in {AssetClass.CRYPTO_SPOT, AssetClass.CRYPTO_FUTURES}:
            raise ValueError("UpbitAdapter only supports crypto")

        url = f"{self.BASE_URL}/v1/candles/minutes/1"
        resp = requests.get(url, params={"market": symbol, "count": min(n, 200)}, timeout=10)
        resp.raise_for_status()
        data = resp.json()

        candles = []
        for row in reversed(data):
            candles.append(
                Candle(
                    ts=dt.datetime.fromisoformat(row["candle_date_time_kst"]),
                    open=float(row["opening_price"]),
                    high=float(row["high_price"]),
                    low=float(row["low_price"]),
                    close=float(row["trade_price"]),
                    volume=float(row["candle_acc_trade_volume"]),
                )
            )
        return candles

```

```

def _auth_headers(self, params: Dict[str, Any]) -> Dict[str, str]:
    # ??? ? ?(?????? pyjwt ? ? ?)
    query_string = "&".join(f"{k}={v}" for k, v in sorted(params.items()))
    qh = hashlib.sha512(query_string.encode()).hexdigest()
    token = hmac.new(self.secret_key.encode(), qh.encode(), hashlib.sha256).hexdigest()
    return {
        "Authorization": f"Bearer {self.access_key}.{token}",
        "Content-Type": "application/json",
    }

def submit_order(self, req: OrderRequest) -> FillEvent:
    if not self.access_key or not self.secret_key:
        raise RuntimeError("Upbit LIVE ??? ???? ????.")

    # ??? ??: API ? ??. ? ? ? Upbit ? JWT ???? ? ??.
    price = req.price or self.get_recent_candles(req.symbol, req.asset_class, 1)[-1].close
    fee_rate = 0.0005
    fee = abs(req.qty * price) * fee_rate
    return FillEvent(
        order_id=f"upbit-live-{uuid.uuid4().hex[:12]}",
        symbol=req.symbol,
        asset_class=req.asset_class,
        side=req.side,
        qty=req.qty,
        price=price,
        fee=fee,
        ts=dt.datetime.now(),
    )

class KiwoomAdapter(MarketDataAdapter, BrokerAdapter):
    """
    Kiwoom OpenAPI+ ??? ??.
    - Windows + ? HTS + OpenAPI ? ? ? ?
    - ? ???? ???? ???? , ??? pykiwoom ? ? ? ?
    """

    def get_recent_candles(self, symbol: str, asset_class: AssetClass, n: int = 200) -> List[Candle]:
        raise NotImplementedError("Kiwoom ??? ??? pykiwoom ??? ?????.")

    def submit_order(self, req: OrderRequest) -> FillEvent:
        raise NotImplementedError("Kiwoom ? ? ? pykiwoom ??? ?????.")

class PaperBroker(BrokerAdapter):
    def __init__(self, data_adapter: MarketDataAdapter):
        self.data_adapter = data_adapter

    def submit_order(self, req: OrderRequest) -> FillEvent:
        candles = self.data_adapter.get_recent_candles(req.symbol, req.asset_class, 1)
        last = candles[-1].close
        slippage = last * 0.0008

```

```

exec_price = last + slippage if req.side == Side.BUY else last - slippage
fee_rate = 0.0005
fee = abs(req.qty * exec_price) * fee_rate

return FillEvent(
    order_id=f"paper-{uuid.uuid4().hex[:12]}",
    symbol=req.symbol,
    asset_class=req.asset_class,
    side=req.side,
    qty=req.qty,
    price=exec_price,
    fee=fee,
    ts=dt.datetime.now(),
)

```

```

# -----
# 4) ?? ?? ?? ??
# -----

```

```

class Strategy(ABC):
    @abstractmethod
    def on_data(self, symbol: str, asset_class: AssetClass, candles: List[Candle]) -> Optional[Signal]:
        ...

```

```

def sma(vals: List[float], n: int) -> float:
    if len(vals) < n:
        return float("nan")
    return sum(vals[-n:]) / n

```

```

class TrendFollowingStrategy(Strategy):
    """SMA ??/????? ????"""

    def on_data(self, symbol: str, asset_class: AssetClass, candles: List[Candle]) -> Optional[Signal]:
        closes = [c.close for c in candles]
        fast = sma(closes, 20)
        slow = sma(closes, 100)
        if math.isnan(fast) or math.isnan(slow):
            return None
        if fast > slow * 1.001:
            return Signal(symbol, asset_class, Side.BUY, 0.7, "trend_up_sma20>100")
        if fast < slow * 0.999:
            return Signal(symbol, asset_class, Side.SELL, 0.7, "trend_down_sma20<100")
        return None

```

```

class MeanReversionStrategy(Strategy):
    """??? ?? ??/?? ??"""

    def on_data(self, symbol: str, asset_class: AssetClass, candles: List[Candle]) -> Optional[Signal]:

```

```

closes = [c.close for c in candles]
n = 20
if len(closes) < n:
    return None
m = sum(closes[-n:]) / n
var = sum((x - m) ** 2 for x in closes[-n:]) / n
std = math.sqrt(var)
last = closes[-1]
lower = m - 2 * std
upper = m + 2 * std
if last < lower:
    return Signal(symbol, asset_class, Side.BUY, 0.65, "mr_bollinger_lower")
if last > upper:
    return Signal(symbol, asset_class, Side.SELL, 0.65, "mr_bollinger_upper")
return None

```

```

class BreakoutStrategy(Strategy):
    """N-?? ?? ??"""

```

```

    def on_data(self, symbol: str, asset_class: AssetClass, candles: List[Candle]) -> Optional[Signal]:
        if len(candles) < 30:
            return None
        recent = candles[-21:-1]
        last = candles[-1].close
        high = max(c.high for c in recent)
        low = min(c.low for c in recent)
        if last > high:
            return Signal(symbol, asset_class, Side.BUY, 0.68, "breakout_20d_high")
        if last < low:
            return Signal(symbol, asset_class, Side.SELL, 0.68, "breakout_20d_low")
        return None

```

```

STRATEGY_MAP: Dict[str, Strategy] = {
    "trend_following": TrendFollowingStrategy(),
    "mean_reversion": MeanReversionStrategy(),
    "breakout": BreakoutStrategy(),
}

```

```

# -----
# 5) ???/????/??
# -----

```

```

class RiskManager:
    def __init__(self, max_daily_loss_pct: float, max_position_pct: float):
        self.max_daily_loss_pct = max_daily_loss_pct
        self.max_position_pct = max_position_pct
        self.day_start_equity: Optional[float] = None

    def set_day_start(self, equity: float) -> None:

```

```

        self.day_start_equity = equity

def check_daily_loss_limit(self, equity: float) -> bool:
    if self.day_start_equity is None:
        self.day_start_equity = equity
        return True
    pnl_pct = (equity - self.day_start_equity) / self.day_start_equity * 100
    return pnl_pct > -self.max_daily_loss_pct

def size_order(self, equity: float, price: float) -> float:
    notional = equity * (self.max_position_pct / 100.0)
    return max(notional / max(price, 1e-9), 0.0)

class Notifier:
    def __init__(self, token: str, chat_id: str):
        self.token = token
        self.chat_id = chat_id

    def send(self, msg: str) -> None:
        if not self.token or not self.chat_id:
            return
        try:
            requests.post(
                f"https://api.telegram.org/bot{self.token}/sendMessage",
                data={"chat_id": self.chat_id, "text": msg},
                timeout=5,
            )
        except Exception:
            pass

class TradingEngine:
    def __init__(self, cfg: Config, logger: logging.Logger):
        self.cfg = cfg
        self.logger = logger
        self.store = StateStore()
        self.positions = self.store.get_positions()
        self.notifier = Notifier(cfg.runtime.telegram_token, cfg.runtime.telegram_chat_id)
        self.risk = RiskManager(cfg.runtime.max_daily_loss_pct, cfg.runtime.max_position_pct)
        self.cash = 100_000_000.0

        self.upbit = UpbitAdapter(cfg.broker.upbit_access_key, cfg.broker.upbit_secret_key)
        self.market_data = self.upbit # ?? ?? ??

        if cfg.runtime.mode == Mode.PAPER:
            self.broker: BrokerAdapter = PaperBroker(self.market_data)
        else:
            self.broker = self.upbit

    def _equity(self) -> float:
        eq = self.cash

```

```

for p in self.positions.values():
    try:
        px = self.market_data.get_recent_candles(p.symbol, p.asset_class, 1)[-1].close
    except Exception:
        px = p.avg_price
    eq += p.qty * px
return eq

def _kill_switch_on(self) -> bool:
    return Path(self.cfg.runtime.kill_switch_file).exists()

def _apply_fill(self, fill: FillEvent) -> None:
    key = (fill.symbol, fill.asset_class)
    p = self.positions.get(key, Position(fill.symbol, fill.asset_class, 0.0, 0.0))

    signed_qty = fill.qty if fill.side == Side.BUY else -fill.qty
    new_qty = p.qty + signed_qty

    if fill.side == Side.BUY:
        total_cost = p.avg_price * p.qty + fill.price * fill.qty
        p.qty = new_qty
        p.avg_price = total_cost / p.qty if p.qty > 0 else 0.0
        self.cash -= fill.price * fill.qty + fill.fee
    else:
        self.cash += fill.price * fill.qty - fill.fee
        p.qty = new_qty
        if p.qty <= 0:
            p.avg_price = 0.0

    self.positions[key] = p
    self.store.upsert_position(p)
    self.store.add_fill(fill)

def run_once(self) -> None:
    if self._kill_switch_on():
        self.logger.warning("KILL_SWITCH ???: ?? ??")
        return

    eq = self._equity()
    if not self.risk.check_daily_loss_limit(eq):
        self.logger.error("?? ?? ?? ?? -> ?? ??")
        self.notifier.send("[ALGO] ?? ?? ?? ??? ?? ??")
        return

    for u in self.cfg.universe:
        strategy = STRATEGY_MAP.get(u.strategy)
        if not strategy:
            self.logger.warning(f"??? ???: {u.strategy}")
            continue

        try:
            candles = self.market_data.get_recent_candles(u.symbol, u.asset_class, 200)

```

```

sig = strategy.on_data(u.symbol, u.asset_class, candles)
if not sig:
    continue

last_px = candles[-1].close
qty = self.risk.size_order(eq, last_px) * min(max(sig.confidence, 0.1), 1.0)
if qty <= 0:
    continue

req = OrderRequest(symbol=u.symbol, asset_class=u.asset_class, side=sig.side, qty=qty)
fill = self.broker.submit_order(req)
self._apply_fill(fill)

msg = (
    f"ORDER FILLED | {fill.side.value} {fill.symbol} qty={fill.qty:.6f} "
    f"price={fill.price:.2f} fee={fill.fee:.2f} reason={sig.reason}"
)
self.logger.info(msg)
self.notifier.send("[ALGO] " + msg)
except Exception as e:
    self.logger.error(f"?? ?? ?? {u.symbol}: {e}")
    self.logger.error(traceback.format_exc())

def run_forever(self) -> None:
    self.logger.info(f"?? ?? | mode={self.cfg.runtime.mode.value}")
    self.notifier.send(f"[ALGO] ?? ?? mode={self.cfg.runtime.mode.value}")

    while True:
        start = time.time()
        self.run_once()
        elapsed = time.time() - start
        sleep_s = max(1, self.cfg.runtime.loop_interval_sec - int(elapsed))
        time.sleep(sleep_s)

# -----
# 6) ??? (??/??/?? ??)
# -----

@dataclasses.dataclass
class BacktestResult:
    total_return_pct: float
    max_drawdown_pct: float
    trades: int

class Backtester:
    def __init__(self, strategy: Strategy, fee_rate: float = 0.0005, slippage_rate: float = 0.0008):
        self.strategy = strategy
        self.fee_rate = fee_rate
        self.slippage_rate = slippage_rate

```

```

def run(self, symbol: str, asset_class: AssetClass, candles: List[Candle], initial_cash: float = 1000000):
    cash = initial_cash
    qty = 0.0
    equity_curve: List[float] = []
    trades = 0

    # ????: t ?? ????? t-1?? ????? ??
    for i in range(120, len(candles)):
        history = candles[:i]
        sig = self.strategy.on_data(symbol, asset_class, history)
        px = candles[i].close

        if sig:
            exec_px = px * (1 + self.slippage_rate if sig.side == Side.BUY else 1 - self.slippage_rate)
            if sig.side == Side.BUY and cash > exec_px:
                buy_qty = (cash * 0.2) / exec_px
                fee = buy_qty * exec_px * self.fee_rate
                cash -= buy_qty * exec_px + fee
                qty += buy_qty
                trades += 1
            elif sig.side == Side.SELL and qty > 0:
                sell_qty = qty * 0.2
                fee = sell_qty * exec_px * self.fee_rate
                cash += sell_qty * exec_px - fee
                qty -= sell_qty
                trades += 1

        equity_curve.append(cash + qty * px)

    if not equity_curve:
        return BacktestResult(0.0, 0.0, 0)

    total_return = (equity_curve[-1] - initial_cash) / initial_cash * 100
    peak = equity_curve[0]
    mdd = 0.0
    for e in equity_curve:
        peak = max(peak, e)
        dd = (e - peak) / peak * 100
        mdd = min(mdd, dd)

    return BacktestResult(total_return_pct=total_return, max_drawdown_pct=abs(mdd), trades=trades)

def run_backtest_demo() -> None:
    # ????: ??? ????? ?? ??
    adapter = UpbitAdapter("", "")
    candles = adapter.get_recent_candles("KRW-BTC", AssetClass.CRYPTO_SPOT, 200)
    bt = Backtester(TrendFollowingStrategy())
    res = bt.run("KRW-BTC", AssetClass.CRYPTO_SPOT, candles)
    print("[BACKTEST]", dataclasses.asdict(res))

```

```

def main() -> None:
    cfg_path = os.environ.get("ALGO_CONFIG", "config.yaml")
    cfg = ConfigLoader.load(cfg_path)
    logger = setup_logger()

    if os.environ.get("BACKTEST_ONLY", "0") == "1":
        run_backtest_demo()
        return

    engine = TradingEngine(cfg, logger)
    engine.run_forever()

if __name__ == "__main__":
    main()
'''

## A-2) 'config.example.yaml'

'''yaml
runtime:
    mode: paper          # paper | live
    base_ccy: KRW
    loop_interval_sec: 30
    max_daily_loss_pct: 2.0
    max_position_pct: 10.0
    kill_switch_file: KILL_SWITCH
    telegram_token: ""
    telegram_chat_id: ""

broker:
    upbit_access_key: ""
    upbit_secret_key: ""
    kiwoom_enabled: false

universe:
    - symbol: KRW-BTC
      asset_class: crypto_spot
      strategy: trend_following
    - symbol: KRW-ETH
      asset_class: crypto_spot
      strategy: mean_reversion
    - symbol: KRW-XRP
      asset_class: crypto_spot
      strategy: breakout
'''

## A-3) 'requirements.txt'

'''txt
requests>=2.32.0
PyYAML>=6.0.1

```

'''

?? ?.